LogicFly: Language-Guided Aerial Navigation with Conditional Task Graphs and Joint Task–Viewpoint Planning

Anonymous Author(s)

***Abstract*—Long-horizon aerial inspection requires efficient travel among multiple targets while respecting task dependencies, acquiring visual evidence, and responding to missing objects. Existing vision-and-language navigation methods often exhibit low success rates on long-horizon, multi-subtask missions, perform redundant actions that reduce execution efficiency, and have limited ability to handle unexpected situations. We present LogicFly, a language-guided aerial navigation framework that combines a reusable hierarchical semantic–geometric map with conditional task reasoning and observation-aware planning. A large language model grounds instructions in the map to construct a Conditional Partial-Order Task Graph (CPTG) that separates mandatory precedence from observation-triggered branches and distinguishes navigation anchors from verification targets. Joint Task–Viewpoint Planning (JTVP) jointly optimizes task order and three-dimensional observation poses within a fixed-size receding horizon. Online verification and language-model-guided semantic-region recovery update task states after negative observations, while safe-corridor-constrained trajectory optimization supports aerial execution. Across 854 simulation episodes in six indoor scenes, LogicFly achieves a 96.96% strict full-task success rate and 0.8394 success weighted by path length. An additional simulation completes all 15 subtasks of a single instruction within the evaluation time budget. Physical UAV experiments further demonstrate multi-target execution with conditional logic and recovery in indoor environments.**

***Note to Practitioners*—LogicFly is applicable to search tasks in a wide range of indoor environments whose structural layouts remain relatively stable. An operator may issue a long natural-language request containing several checks, mandatory precedence relations, freely ordered subtasks, and observation-dependent alternatives. The framework converts such requests into a conditional partial-order task graph and jointly plans task order and three-dimensional observation viewpoints while respecting task dependencies and conditional branches. Negative visual evidence can trigger bounded semantic-region search and replanning rather than immediate failure. This coordinated planning reduces flight distance while enabling the UAV to acquire task-specific visual evidence. Deployment requires a previously constructed semantic–geometric map, calibrated RGB-D sensing, and localization in the map frame. The demonstrated platform uses an NVIDIA Jetson Orin NX (16 GB), a PX4 flight controller, a Livox Mid-360 LiDAR, and an Intel RealSense D435 depth camera, with FAST-LIO localization. Performance depends on map quality, perception reliability, and the coverage of candidate viewpoints; changes that invalidate the stored map may require renewed exploration or online recovery.**

***Index Terms*—Aerial robotics, task and motion planning, vision-and-language navigation, semantic mapping.**

I. Introduction

As unmanned aerial vehicles (UAVs) are increasingly deployed in intelligent building inspection, indoor security, and emergency response, enabling autonomous navigation guided by complex natural-language instructions has become an important research challenge. Real-world missions often involve multiple spatial regions and target objects, subject to ordering constraints, conditional dependencies, and dynamic task requirements. Successfully executing such missions requires UAVs to go beyond conventional perception and motion planning by integrating spatial-semantic representations, efficient multi-goal task planning, and logical reasoning. Effective task planning is also essential to minimize redundant travel and maximize mission efficiency within limited flight time. Moreover, visual occlusions, uncertain target locations, and environmental changes demand adaptive decision-making based on real-time observations, rather than reliance on predefined task sequences. Vision-and-Language Navigation (VLN) benchmarks such as R2R [1] and VLN-CE [2] have advanced instruction following in discrete and continuous environments, while ALFRED [3] and long-horizon VLN [4] extend embodied reasoning toward compositional tasks.

Existing methods leave three gaps for multi-target aerial inspection. First, sequential multi-goal navigation prescribes goal order [5], [6], limiting opportunities to shorten routes by reordering independent tasks. Temporal-logic planners accommodate richer constraints [7], but these must also be connected to visual verification. Second, semantic maps ground spatial targets [8], [9], and visibility-aware methods select observation viewpoints [10]; selecting views separately from task order can nevertheless overlook shorter routes through alternative valid views. Third, feedback-based replanning and persistent semantic memory support adaptation [11]–[13], but do not by themselves specify how unsuccessful observations should coordinate viewpoint retries, alternative-region search, and conditional-task updates.

To address these challenges, we propose **LogicFly**, a language-guided aerial navigation framework for long-range, multi-subtask instructions with complex logical dependencies, as illustrated in Fig. 2. LogicFly grounds instructions in a reusable hierarchical 3D semantic–geometric map constructed through autonomous exploration, representing free space, floors, rooms, cross-floor transitions, and open-vocabulary object instances. Building on this representation, three coordinated designs address the above challenges. First, a large language model constructs a **Conditional Partial-Order Task Graph (CPTG)** that represents mandatory precedence, freely schedulable subtasks, and observation-triggered branches while distinguishing navigation anchors from ver-

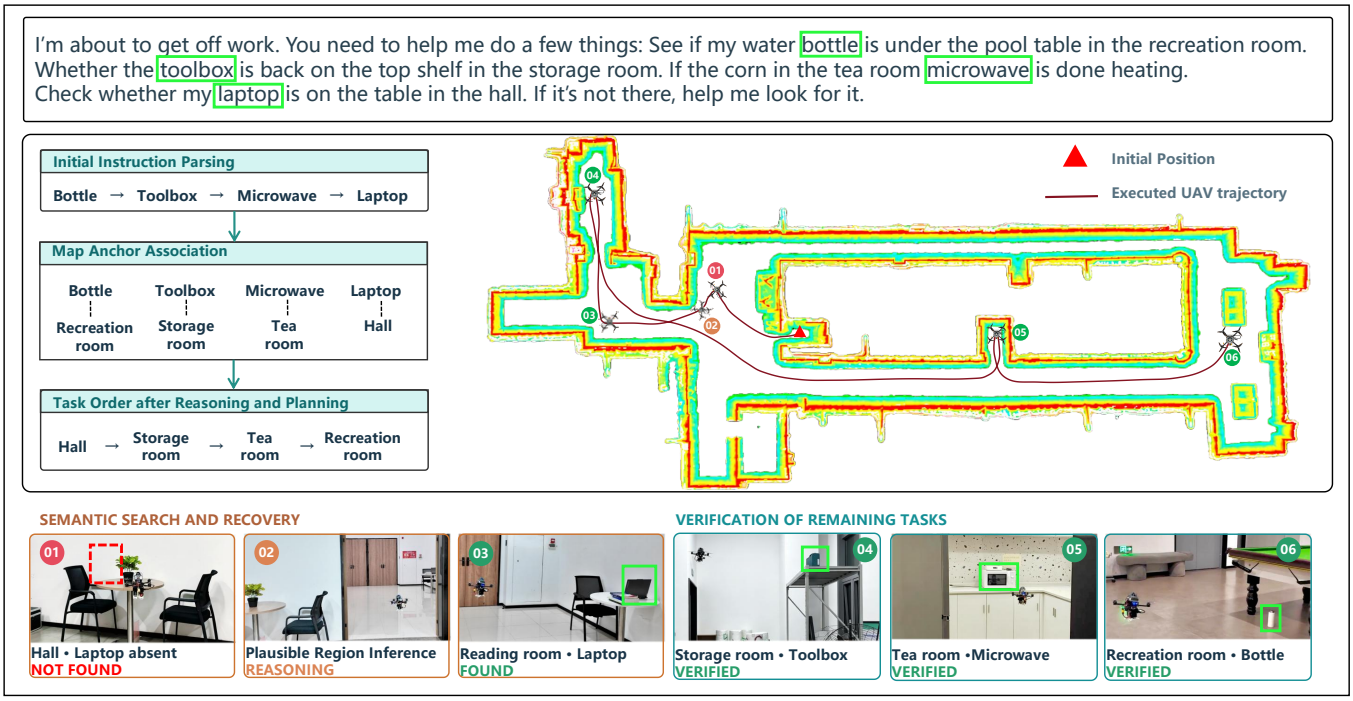

Fig. 1. Real-world execution of a multi-target instruction by LogicFly. The UAV first fails to detect the laptop at its instructed location, performs semantic recovery, and then verifies the laptop, toolbox, microwave state, and water bottle. The map panel shows the executed route, while numbered views document the recovery and task-verification sequence.

ification targets. Second, **Joint Task–Viewpoint Planning (JTVP)** jointly selects task order and collision-free 3D observation poses within a receding horizon, accounting for travel cost, target visibility, and observation viewpoint quality. Third, an **observation-driven autonomous recovery mechanism** retries local viewpoints and, after local search is exhausted, uses language-model-guided semantic-region hypotheses and negative-evidence updates to redirect the search. Online verification updates task states and conditional branches, informing subsequent planning decisions. Safe-corridor-constrained trajectory optimization supports aerial execution, while spatially consistent multi-view detections enrich the semantic map after execution for subsequent missions.

We evaluate LogicFly on 854 valid episodes generated from 152 instruction cases across six HM3D scenes. The benchmark includes single- and multi-target tasks, single- and cross-floor navigation, precedence constraints, conditional branches, spatial relations, verified absence, semantic recovery, and target disambiguation. LogicFly achieves a 97.0% strict full-task success rate and an SPL of 0.84, including 96.70% success on multi-target tasks. Ablation studies show that CPTG is critical to reliable multi-target execution, while JTVP increases SPL by 5.33 percentage points and reduces mean successful-task completion time by approximately 6.6% at the same success rate. We further deploy LogicFly on a physical UAV in multiple real indoor environments, demonstrating reliable execution of long-range, multi-subtask navigation instructions with complex logical dependencies.

The main contributions of this paper are summarized as follows:

- We present **LogicFly**, a language-guided aerial navigation framework that reuses an autonomously constructed hierarchical semantic–geometric representation to execute long-range instructions containing multiple subtasks and complex logical dependencies.
- We propose a **Conditional Partial-Order Task Graph (CPTG)** that explicitly represents mandatory precedence, freely schedulable tasks, observation-triggered branches, and distinct navigation and verification targets.
- We develop **Joint Task–Viewpoint Planning (JTVP)** to jointly optimize task order and three-dimensional observation viewpoints, improving travel efficiency while acquiring task-specific visual evidence.
- We introduce an **observation-driven autonomous recovery mechanism** that combines local viewpoint retries, language-model-guided search over alternative semantic regions, and negative-evidence updates to resume target search after unsuccessful observations.
- We conduct extensive **simulation and real-world evaluations**, demonstrating high strict full-task success, improved efficiency over representative baselines, and reliable deployment on a physical UAV in complex indoor tasks.

II. Related Work

A. Long-Horizon and Multi-Target Vision–Language Navigation

Early VLN benchmarks study route following [1], [14], object grounding [15], [16], and continuous-space navigation [2]. PREVALENT [17] improves multimodal pre-training, while ScaleVLN [18] scales navigation training through large-scale instruction–trajectory generation. These advances improve navigation performance within established VLN task formulations, rather than extending them to inspection-task scheduling. HAMT [19] and DUET [20] encode temporal or graph-structured history to inform navigation decisions; these representations primarily support navigation-history reasoning, rather than explicitly modeling precedence constraints and scheduling decisions among independent inspection tasks.

For longer instructions, M-TRACK [21] checks instruction-derived milestones before advancing, addressing premature transitions between subtasks. Its sequential progression, however, does not select among alternative orders of independent checks. LH-VLN [4] uses dynamic memory to support consecutive subtasks, but likewise emphasizes execution consistency along a task sequence. MultiON [5] explicitly evaluates multiple object goals, yet fixes their order, preventing the agent from shortening travel by rearranging goals. GOAT-Bench [6] broadens sequential navigation to multimodal open-vocabulary goals, while retaining a sequence-based task interface. Complementarily, Iterative VLN [22] studies experience reuse across episodes, and OneMap [13] retains semantic observations across searches. While these methods improve experience reuse and semantic memory retention, they do not explicitly represent task precedence constraints or observation-triggered branches for inspection-task scheduling. LogicFly uses CPTG to represent mandatory dependencies and observation-triggered branches while preserving ordering freedom among independent subtasks.

B. Semantic–Geometric Memory and Zero-Shot Navigation

3D Dynamic Scene Graphs [23] and Hydra [24] organize geometry and semantic entities hierarchically, while OpenScene [25] supports open-vocabulary queries over 3D features. VLMaps [8] connects language queries to spatial navigation goals. ConceptGraphs [26] represents open-vocabulary objects and relations, and HOV-SG [9] organizes floors, rooms, and objects for hierarchical querying. Although these representations support semantic scene understanding and object localization, they do not by themselves guarantee a viewpoint suitable for visual verification of a retrieved target.

Navigation methods use pretrained models to turn observations and spatial context into decisions. LM-Nav [27] composes language, vision, and navigation models, and L3MVN [28] uses language-model commonsense for frontier selection. NavGPT [29] reasons over observations and history; MapGPT [30] adds map-guided multi-step planning. SpatialGPT [31] and SpatialNav [32] use structured spatial memory or scene graphs, while VLN-Zero [33] and Open-Nav [34] study scene-graph-assisted exploration and continuous-space zero-shot navigation, respectively. These navigation formulations do not jointly select a schedule of independent inspection tasks and their observation poses. MSGNav [10] explicitly addresses the observation-pose issue by sampling and scoring viewpoints for target visibility. However, its viewpoint selection is primarily target-centric, without explicitly jointly optimizing inspection-task ordering and viewpoint selection across multiple targets. LogicFly jointly optimizes inspection-task ordering and 3D observation poses using collision-aware travel costs and task-specific viewpoint constraints and quality scores.

C. Language-Grounded Task and Motion Planning

SayPlan [11] combines hierarchical semantic search, language-model planning, and a classical planner, using simulator feedback to revise infeasible plans. While simulator feedback helps identify and revise infeasible plans, the framework does not explicitly formulate inspection-specific visual evidence as a planning constraint. OSGP [7] explicitly handles task constraints by translating natural-language missions into linear temporal logic and performing hierarchical search with LLM guidance. Its logical planning formulation does not jointly select alternative visual verification poses for inspection tasks. Ray *et al.* [35] address planning scale through sparse scene-graph-derived task-and-motion domains and incremental inclusion of relevant objects. This reduces computation on irrelevant scene elements, but domain sparsification does not itself define a search policy for targets missing from their expected locations.

Inner Monologue [12] incorporates success detection and scene feedback into language-model planning, allowing decisions to change after execution failures. Its feedback-driven replanning does not explicitly track geometric search coverage to distinguish newly inspected areas from repeated observations. For target search, this distinction matters because repeated views of the same area should not be treated as additional coverage. LogicFly uses coverage-aware negative-evidence updates to revise regional confidence, linking local viewpoint retries and alternative-region search with downstream conditional-task updates.

III. System Overview

Figure 2 illustrates the pipeline of LogicFly, which consists of map construction, instruction grounding, joint planning, and closed-loop execution.

Map construction. The UAV first explores the environment and builds a reusable semantic–geometric map containing navigable space, rooms, and objects.

Instruction grounding. Given a natural-language instruction and the map, a large language model constructs a CPTG that specifies the subtasks, their dependencies, and observation-dependent branches.

Joint planning. JTVP selects the order and observation poses of currently executable tasks. The selected route is converted into a trajectory for UAV execution.

Execution and recovery. The UAV executes the first planned visit and verifies the task using onboard observations.

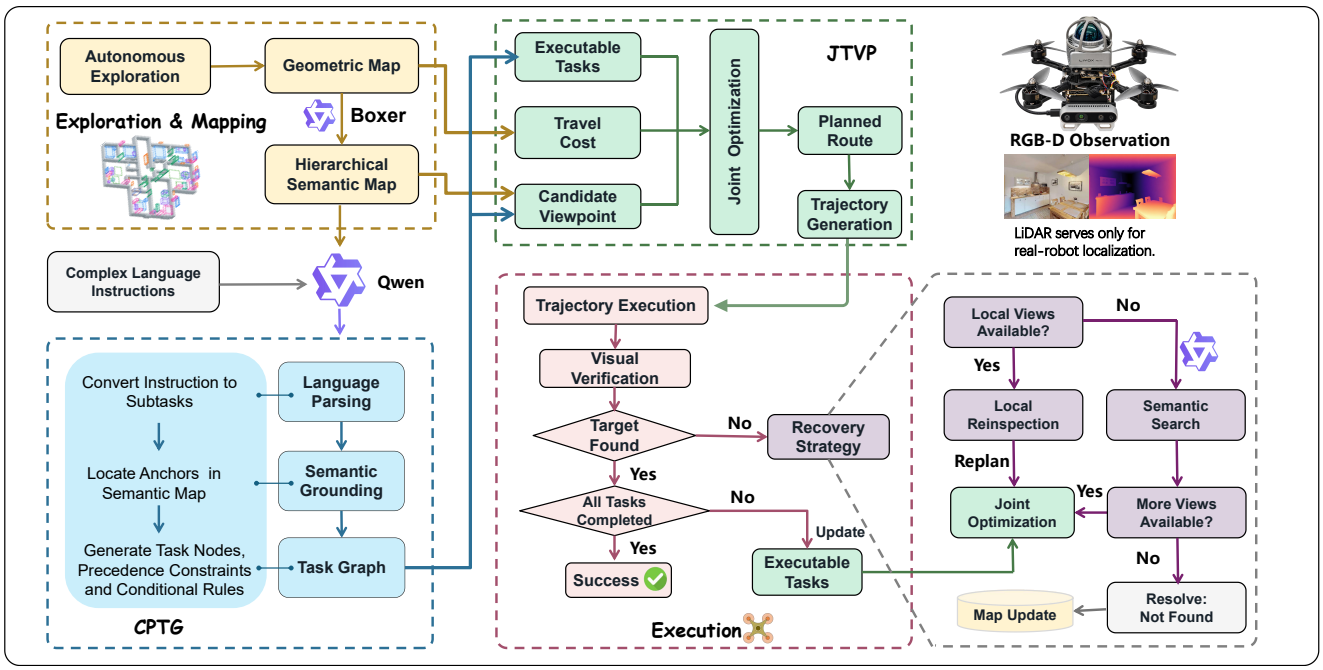

Fig. 2. Overview of LogicFly. A natural-language instruction and an autonomously explored hierarchical semantic–geometric representation are converted into a CPTG and an executable task set. JTVP jointly selects task order and 3D observation poses within a receding horizon. The first planned visit is then executed and verified online; the resulting observations update the task and search states, revise semantic map priors, and trigger recovery and replanning when necessary.

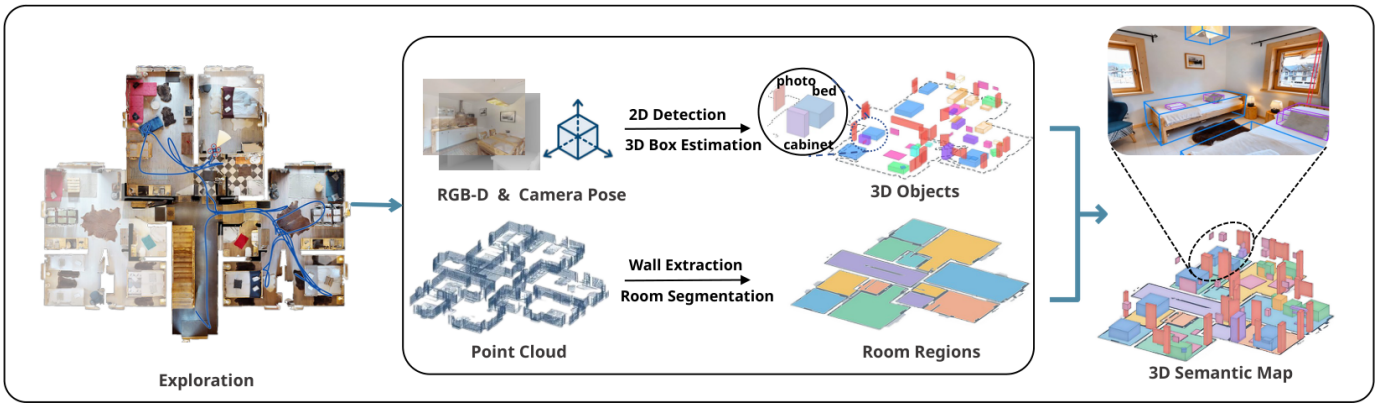

Fig. 3. Construction of the hierarchical semantic–geometric map. RGB-D observations provide object instances, while point-cloud processing yields room regions; these are integrated with exploration data in the 3D map.

Verification outcomes update the task graph. When permitted by the task policy, unsuccessful observations trigger local reinspection or search in alternative semantic regions. The updated task and search states feed back into planning until the instruction is completed or a stopping condition is reached.

IV. METHODOLOGY

A. Reusable 3D Semantic–Geometric Map

The robot first explores an initially unknown environment and constructs a reusable hierarchical semantic–geometric representation from onboard observations. The geometric component $\mathcal{M}_g$ records occupied, observed-free, and unknown 3D space, while the semantic component $\mathcal{M}_s$ organizes floors, rooms, cross-floor transitions, and open-vocabulary object instances in a shared global frame. We use FALCON [36] for autonomous aerial exploration; the resulting trajectories support the acquisition of RGB-D observations, camera poses, and occupancy evidence. The updated perception and room modules below convert these observations into auditable object instances and supported room regions without filling unknown space as free or silently assigning it to a room. This representation supplies semantic anchors for task grounding and geometric free space for navigation. Figure 3 summarizes its construction from exploration data.

1) Target Recognition and Multi-View 3D Fusion: Target recognition and multi-view fusion convert RGB-D observations into spatially consistent object instances for language grounding.

The mapping front end uses text-prompted SAM 3.1 in image mode rather than video tracking. It applies a fixed indoor

vocabulary and a confidence threshold specified in Experimental Setup. Each RGB frame returns a 2D proposal, category, confidence, and instance mask. The mask preserves foreground depth only inside its proposal footprint and suppresses box-background depth before calibrated RGB-D pixels are lifted into the world frame. Boxer [37] then estimates the oriented 3D instance

$$\mathbf{o}_{t,k} = \big(\mathbf{c}_{t,k}, \mathbf{s}_{t,k}, \mathbf{R}_{t,k}, \ell_{t,k}, p_{t,k}\big), \tag{1}$$

In (1), t is the exact source-frame identity; k is the proposal index; $\mathbf{c}$, $\mathbf{s}$, and $\mathbf{R}$ are the center, size, and orientation, respectively; and ℓ and p are the category and confidence.

Multi-view fusion first applies class-consistent non-maximum suppression and then associates proposals using 3D overlap and semantic agreement. Exact capture timestamps are retained as frame identities, and at most the highest-confidence proposal from each frame contributes to one fused instance. The association thresholds and minimum independent-frame requirement are fixed in Experimental Setup. Category names and semantic IDs are jointly tie-broken deterministically. Let $\mathcal{I}_j$ be the proposals associated with fused instance j, and let $\mathcal{T}_j = \{t \mid \exists k : (t,k) \in \mathcal{I}_j\}$ be their distinct source frames. The confidence-weighted center is computed as

$$\widehat{\mathbf{c}}_j = \frac{\sum_{(t,k)\in\mathcal{I}_j} p_{t,k}\mathbf{c}_{t,k}}{\sum_{(t,k)\in\mathcal{I}_j} p_{t,k}}, \qquad |\mathcal{T}_j| \geq n_{\text{mv}}. \tag{2}$$

The fused instance obtained from (2) becomes the semantic anchor used by room assignment and subsequent language tasks. During online verification, detections enter through the same RGB-D projection interface; known instances use their map IDs, while an unmatched same-category detection becomes a novel online track only after satisfying the minimum-frame and spatial-consistency settings specified in Experimental Setup. Together, these steps produce object instances with consistent semantic identities and 3D locations for room assignment and task grounding.

2) Wall- and Observation-Supported Room Partitioning: Room partitioning combines measured wall barriers with observation support to define room regions before assigning semantic categories.

Room partitioning is performed independently on a horizontal grid for each floor. Wall points extracted from measured geometry are projected into a barrier set $\mathcal{W}$; a bounded local morphological closing only bridges sampling gaps and never hallucinates unknown wall points. The grid resolution and closing neighborhood are fixed in Experimental Setup. Let $\mathcal{G}$ denote the horizontal grid and let $\Omega_0 \subseteq \mathcal{G}$ be the candidate floor domain. To prevent walls or interpolation holes from becoming room evidence, we combine multiple independent support sets: near-horizontal measured surfaces $\mathcal{S}_h$, multi-height free-space observations $\mathcal{S}_f$, multi-frame depth-ray coverage $\mathcal{S}_r$, and recorded trajectory coverage $\mathcal{S}_t$. Each support set is projected onto $\mathcal{G}$ and represented as a set of grid cells. The effective partition domain is defined as

$$\Omega = (\Omega_0 \setminus \mathcal{W}) \cap \big(\mathcal{S}_h \cup \mathcal{S}_f \cup \mathcal{S}_r \cup \mathcal{S}_t\big). \tag{3}$$

According to (3), Ω contains cells that are not blocked by walls and have support from at least one evidence source.

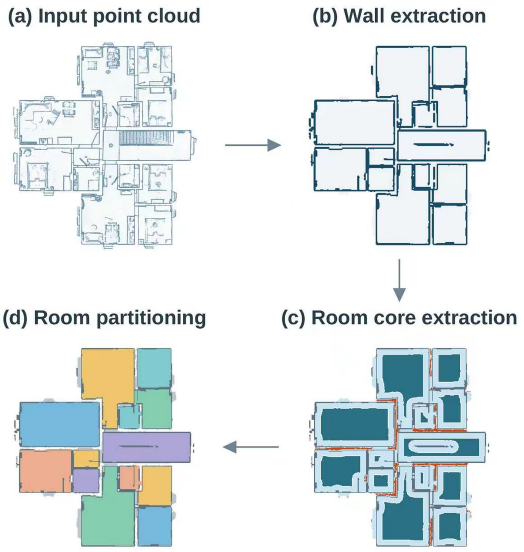

Fig. 4. Room-partitioning pipeline. The input point cloud is processed to extract walls, identify room cores, and produce the final room partition.

A cell without independent support is excluded from the room partition but is not labeled as exterior or occupied; flight feasibility remains determined by the independent 3D occupancy map.

On Ω, we compute the Euclidean distance transform $d(\mathbf{x})$ to the domain boundary. Broad room interiors have larger clearance than doorways, corridors, and room boundaries. We therefore define the interior cores as

$$\mathcal{C} = \{\mathbf{x} \in \Omega \,|\, d(\mathbf{x}) > \tau_c\}, \tag{4}$$

where τ_c is the interior-clearance threshold in (4) and is specified in Experimental Setup. Connected components smaller than A_{min} are discarded, with A_{min} fixed in Experimental Setup. Each retained core is a spatial marker rather than a semantic room label. Marker-controlled watershed propagation runs on $-d$ with grid-connected, multi-source priority-queue flooding and is restricted to Ω; it therefore stops at wall barriers and unsupported boundaries, while low-clearance connections become bottlenecks between competing cores. A small enclosed region is merged into its sole labeled neighbor only when its boundary wall support is weak and it does not border unsupported open space. A supported connected component without a marker remains untyped rather than receiving an artificial seed.

Room categories are assigned only after geometry: measured RGB frames whose camera positions lie in a region are paired with that region's fused 3D objects and passed to a multimodal language model acting as a classifier over a fixed set of room categories. The frame-count setting is given in Experimental Setup. Insufficient evidence yields "unknown", and the classifier cannot change the room boundary or occupancy map. Figure 4 summarizes the room-partitioning pipeline, from the input point cloud through wall and room-core extraction to the final partition. The resulting room regions group fused

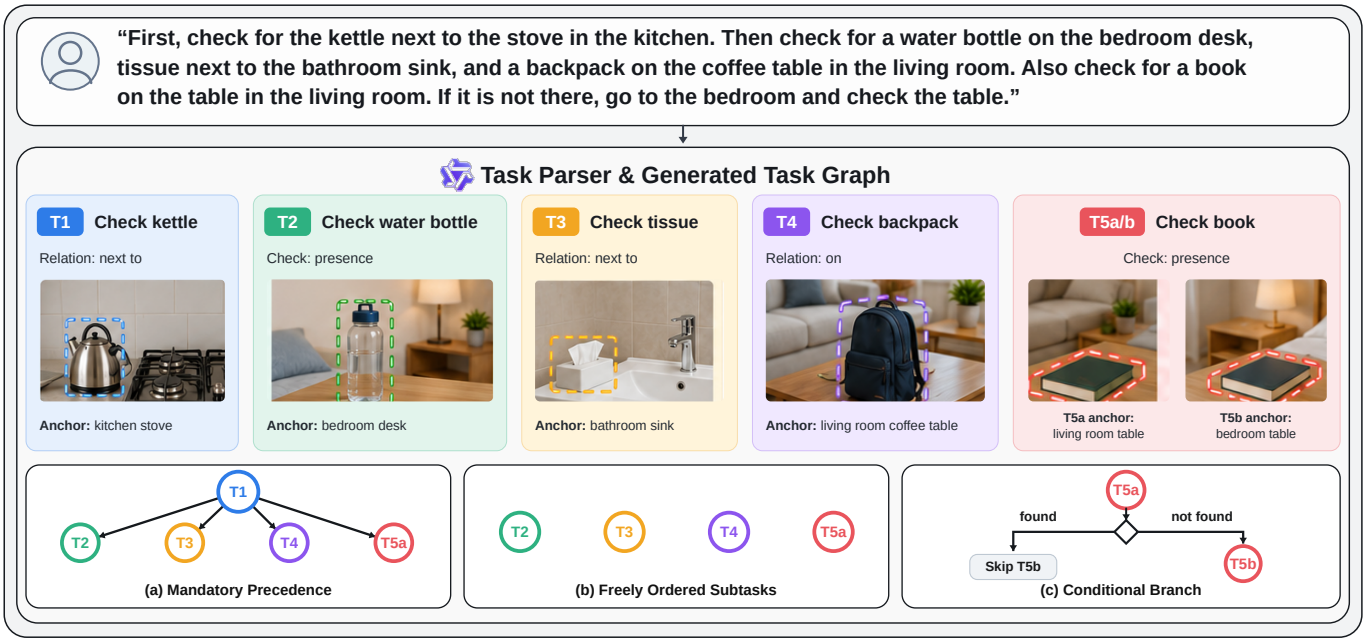

Fig. 5. From a natural-language instruction to a CPTG. The task parser constructs task nodes with navigation anchors, semantic targets, and verification predicates; precedence and conditional edges encode mandatory ordering and observation-dependent activation.

object instances under semantic categories while preserving the independently measured occupancy geometry.

Together, object fusion and room partitioning form the reusable semantic hierarchy, while the geometric map provides free-space and connectivity information for task grounding, planning, and recovery.

B. Conditional Partial-Order Task Graph

CPTG represents the instruction as subtasks with mandatory dependencies and observation-dependent branches, providing the executable task set for planning.

Given a natural-language instruction I, the system constructs a **Conditional Partial-Order Task Graph**

$$\mathcal{G}_T = (\mathcal{V}, \mathcal{E}_{\text{prec}}, \mathcal{R}_{\text{cond}}), \tag{5}$$

where $\mathcal{V}$ denotes language-grounded subtasks, $\mathcal{E}_{\text{prec}}$ specifies mandatory precedence relations, and $\mathcal{R}_{\text{cond}}$ contains observation-dependent activation and skip rules. Each task in (5) stores its action, semantic target, navigation anchor, verification predicate, spatial constraints, room/floor scope, prerequisites, and search policy. The language model produces these fields as structured JSON from the instruction and map inventory. Before execution, the graph is rejected if entity references are invalid, dependencies are inconsistent, or the precedence relation contains a cycle. Figure 5 illustrates the resulting task representation.

For each task, the **navigation anchor** defines the observation region, the **semantic target** specifies the entity to detect, and the **verification predicate** defines the requested evidence. Spatial relations constrain grounding and candidate generation; their satisfaction is recorded geometrically and required by the strict full-task evaluator.

Let $\mathcal{R}_t$ denote tasks whose task-level outcomes have been resolved and $\mathcal{A}_t$ the active pending tasks. The currently executable set is

$$\mathcal{E}_t = \{ i \in \mathcal{A}_t \,|\, \text{pred}(i) \subseteq \mathcal{R}_t \}. \tag{6}$$

Resolved outcomes are interpreted by $\mathcal{R}_{\text{cond}}$ to activate or skip downstream conditional tasks. Thus, only tasks satisfying (6) enter the planner. Each conditional source is resolved according to its observation policy, either at the first designated viewpoint or after a terminal search outcome. Separating mandatory precedence from conditional activation prevents observation-dependent alternatives from being incorrectly imposed as unconditional order constraints. CPTG thus determines the executable tasks, while JTVP plans their order and observation poses.

C. Task-Conditioned Observation and Motion Modeling

Observation and motion modeling converts executable tasks into feasible camera poses and estimates the cost of traveling between them. Task-specific visibility and spatial constraints determine candidate views, while geometric path queries provide transition costs for joint planning.

For each executable task i, its navigation anchor and spatial constraints define a task-specific 3D observation region, such as an object, support surface, surrounding area, lower region, or relation-defined area.

The candidate observation pose indexed by k is $\mathbf{q}_i^k = (x_i^k, y_i^k, z_i^k, \psi_i^k)$, where (x_i^k, y_i^k, z_i^k) is the camera position and ψ_i^k is yaw. Candidates are sampled at multiple distances, azimuths, and feasible heights, with yaw oriented toward the observation region; vertical visibility is handled by candidate height and the camera field of view.

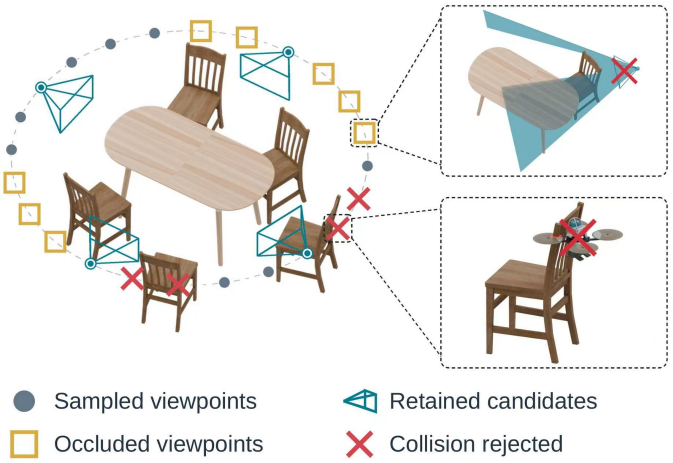

Fig. 6. Viewpoint sampling and filtering for a dining-table target. Sampled poses are rejected when occluded or collision-prone; the remaining feasible views are retained for observation.

Let $\mathcal{P}_i$ denote samples from the observation region and $\mathcal{V}_i(\mathbf{q}) \subseteq \mathcal{P}_i$ the samples visible from $\mathbf{q}$ under camera field-of-view and line-of-sight constraints. We define the geometric coverage as

$$Q_i(\mathbf{q}) = \frac{|\mathcal{V}_i(\mathbf{q})|}{\max\{1, |\mathcal{P}_i|\}}. \tag{7}$$

Candidates violating occupancy, flight-height, spatial-relation, or required-visibility constraints are discarded. The remaining poses are scored using the coverage in (7):

$$\begin{aligned} C_i^{\text{view}}(\mathbf{q}) = \beta_d \frac{|d - d_i^{\text{pref}}|}{\ell_0} + \beta_Q \big[1 - Q_i(\mathbf{q})\big] \\ + \beta_z \frac{|z - \bar{z}_i|}{\ell_0} + \beta_r \frac{d_i^{\text{ref}}(\mathbf{q})}{\ell_0} + \beta_b \frac{d_i^{\text{between}}(\mathbf{q})}{\ell_0}. \end{aligned} \tag{8}$$

Here, d is the sampled horizontal standoff from the anchor's bounding footprint, d_i^{pref} is its preferred value, z is the candidate height, and $\bar{z}_i$ is the mean observation-region height. Let $a(\mathbf{q})$ identify the anchor that generated a candidate, $\mathbf{c}_{a,xy}$ its horizontal center, $\mathbf{x}(\mathbf{q}) = (x, y)$ the candidate's horizontal position, and $\mathcal{B}_i$ the instruction's reference objects. The relation terms, measured in meters, are

$$\begin{aligned} d_i^{\text{ref}}(\mathbf{q}) &= \min_{b \in \mathcal{B}_i} \|\mathbf{c}_{a(\mathbf{q}),xy} - \mathbf{c}_{b,xy}\|_2, \\ d_i^{\text{between}}(\mathbf{q}) &= \text{dist}\big(\mathbf{x}(\mathbf{q}), [\mathbf{c}_{b_1,xy}, \mathbf{c}_{b_2,xy}]\big). \end{aligned} \tag{9}$$

The first term is zero when $\mathcal{B}_i$ is empty; it compares alternative anchors and is constant across views of the same anchor. The second term is used only for a "between" relation with two distinct reference centers and is the distance to their closed line segment. Let $\mathbf{h} = \mathbf{c}_{b_2,xy} - \mathbf{c}_{b_1,xy}$ and $u = (\mathbf{x}(\mathbf{q}) - \mathbf{c}_{b_1,xy})^\top \mathbf{h}/\|\mathbf{h}\|_2^2$ be the normalized projection parameter; the hard filter requires $u \in [-0.05, 1.05]$ and segment distance at most $\max(0.75\,\text{m}, d_{i,\max}/2)$, where $d_{i,\max}$ is the maximum sampling standoff. Coincident reference centers are rejected. Other inapplicable terms are zero. The reference length ℓ_0 makes (8) dimensionless. Hard feasibility is enforced before scoring, so a low soft cost cannot compensate for an unsafe or unobservable pose. All weights are nonnegative and given in Section V-A. The retained set $\mathcal{Q}_i = \{\mathbf{q}_i^1, \ldots, \mathbf{q}_i^{K_i}\}$ preserves diverse feasible viewpoints.

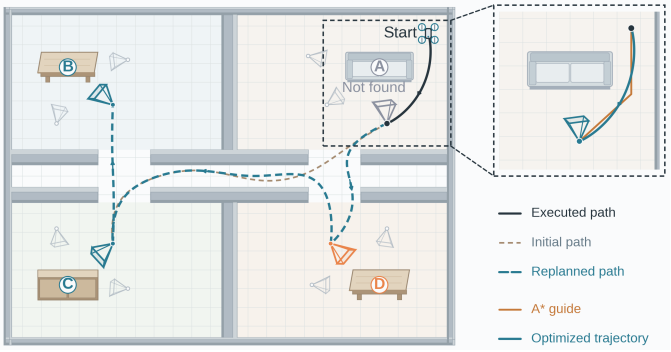

Fig. 7. Joint task–viewpoint planning and trajectory refinement. After an observation returns no target, the planner replans and produces a collision-free trajectory guided by A*.

Figure 6 illustrates the sampling and feasibility-filtering process.

For candidate poses $\mathbf{q}_a$ and $\mathbf{q}_b$, a 3D A* query returns a collision-free polyline $P_t(\mathbf{q}_a, \mathbf{q}_b) = \{\mathbf{p}_1, \ldots, \mathbf{p}_N\}$ when the transition is reachable. Let $\Delta\psi_{ab} = \text{wrap}(\psi_b - \psi_a)$. The path length and motion-time proxy are defined, respectively, as

$$L_t(\mathbf{q}_a, \mathbf{q}_b) = \sum_{u=1}^{N-1} \|\mathbf{p}_{u+1} - \mathbf{p}_u\|_2, \tag{10a}$$

$$\widehat{T}_t(\mathbf{q}_a, \mathbf{q}_b) = \max\left\{ \frac{L_t(\mathbf{q}_a, \mathbf{q}_b)}{v_{\text{nom}}}, \frac{|z_b - z_a|}{v_z}, \frac{|\Delta\psi_{ab}|}{\omega_\psi} \right\}. \tag{10b}$$

Here, $v_{\text{nom}} > 0$ is a nominal speed along the three-dimensional polyline in (10b), while $v_z > 0$ and $\omega_\psi > 0$ set nominal climb and yaw rates. The wrap operator returns an angle in $[-\pi, \pi)$. The maximum models concurrent translation and yaw as a ranking heuristic, not the duration of the optimized trajectory. Cross-floor detours and intermediate ascents or descents contribute to L_t; the second term accounts only for net altitude change. Acceleration, intermediate vertical-rate constraints, and observation dwell are handled during execution rather than predicted by this proxy. Unreachable transitions have $L_t = \widehat{T}_t = +\infty$.

Figure 7 shows A*-guided trajectory refinement after a negative observation, including route replanning and trajectory optimization.

The resulting candidate sets and observation scores describe where each task can be inspected, while path lengths and motion-time estimates quantify travel between candidates. These quantities provide the inputs to JTVP.

D. Receding-Horizon Joint Task–Viewpoint Planning

JTVP jointly chooses task order and observation poses within a bounded task window. It uses subset dynamic programming to minimize the combined travel and observation cost, then executes the first visit before updating the plan.

At each planning cycle, the planner selects a local task window $\mathcal{W}_t \subseteq \mathcal{E}_t$ with $|\mathcal{W}_t| \leq H$. Tasks are prioritized by their best candidate's current-position distance plus terminal cost, with task IDs breaking ties; recovery can focus the

window on the task under reinspection. Every predecessor of a task in $\mathcal{E}_t$ is already resolved. Thus, no additional precedence constraint is needed within this window; newly enabled successors enter a later planning cycle.

Candidate poses are grouped by physical location and prioritized using current-pose distance, motion-time estimate, and observation score. The retained set $\widetilde{\mathcal{Q}}_{i,t} \subseteq \mathcal{Q}_i$ maintains representative viewpoints across semantic locations and is progressively augmented when alternatives are needed.

Given $m = |\mathcal{W}_t| > 0$, let $\Pi(\mathcal{W}_t)$ be all permutations of the window and $\mathcal{K}_{i,t} = \{k : \mathbf{q}_i^k \in \widetilde{\mathcal{Q}}_{i,t}\}$. JTVP jointly selects $\pi \in \Pi(\mathcal{W}_t)$, visiting each task exactly once, and one observation pose $\mathbf{q}_r = \mathbf{q}_{\pi_r}^{k_r}$ for each selected task, with $\mathbf{q}_0 = \mathbf{q}_t^{\text{robot}}$. Let $\iota(\mathbf{q})$ denote the task associated with candidate $\mathbf{q}$. We define the dimensionless transition score as

$$c_t(\mathbf{q}_a, \mathbf{q}_b) = \eta_L \frac{L_t(\mathbf{q}_a, \mathbf{q}_b)}{\ell_0} + \eta_T \frac{\widehat{T}_t(\mathbf{q}_a, \mathbf{q}_b)}{\tau_0} + \eta_V C^{\text{view}}_{\iota(\mathbf{q}_b)}(\mathbf{q}_b), \tag{11}$$

where τ_0 is a reference time. Using (11), JTVP solves

$$\begin{aligned} \underset{\pi, \mathbf{k}}{\text{minimize}} \quad & \sum_{r=1}^{m} c_t(\mathbf{q}_{r-1}, \mathbf{q}_r) \\ \text{subject to} \quad & \pi \in \Pi(\mathcal{W}_t), \quad k_r \in \mathcal{K}_{\pi_r,t}, \\ & \mathbf{q}_r = \mathbf{q}_{\pi_r}^{k_r}, \quad r = 1, \dots, m, \\ & L_t(\mathbf{q}_{r-1}, \mathbf{q}_r) < \infty, \quad r = 1, \dots, m. \end{aligned} \tag{12}$$

The joint planning problem in (12) is solved using subset dynamic programming. Let $D(S, i, k)$ be the minimum cost of visiting exactly $S \subseteq \mathcal{W}_t$ and ending at $\mathbf{q}_i^k$, where $i \in S$ and $k \in \mathcal{K}_{i,t}$. Invalid or unreachable states have value $+\infty$, as do unreachable transition scores. For nonempty S and $j \in \mathcal{W}_t \setminus S$, the initialization, recurrence, and terminal cost are

$$D(\{i\}, i, k) = c_t(\mathbf{q}_0, \mathbf{q}_i^k), \tag{13a}$$

$$D(S \cup \{j\}, j, \ell) = \min_{i \in S,\, k \in \mathcal{K}_{i,t}} \left[D(S, i, k) + c_t(\mathbf{q}_i^k, \mathbf{q}_j^\ell) \right], \tag{13b}$$

$$J_t^\star = \min_{i \in \mathcal{W}_t,\, k \in \mathcal{K}_{i,t}} D(\mathcal{W}_t, i, k). \tag{13c}$$

The recurrence is evaluated for $\ell \in \mathcal{K}_{j,t}$; a minimum over an empty set is $+\infty$. An empty window yields an empty plan with zero cost: execution finishes only if all required outcomes are resolved, otherwise it reports unresolved tasks. Empty candidate sets or an infinite terminal cost trigger expansion from representative views to the full available pool. If no feasible plan is found, planning terminates with failure rather than marking the task verified. For a finite terminal cost, backtracking through (13) returns the optimal order and poses for the selected window and candidate sets. Only the first visit is executed before replanning from the updated robot pose, task state, candidate set, and semantic evidence. This receding-horizon policy preserves the efficiency of multi-task lookahead without committing later visits to assumptions that subsequent observations may invalidate. With $K = \max_{i \in \mathcal{W}_t} |\mathcal{K}_{i,t}|$ and cached transition costs, the subset DP uses $O(2^m m^2 K^2)$ time and $O(2^m m K)$ memory. Since $m \le H$, the exponential dependence is limited to the local window. Task screening, candidate generation, and A* queries still depend on the available tasks and map size; retaining views per location does not impose a global constant K when additional locations are admitted. The 15-subtask timing in Section V-B measures the complete planner under this policy.

JTVP therefore returns a jointly selected task order and observation poses for the current window, with the first visit passed to trajectory generation and later visits reconsidered after new observations.

E. Safe-Corridor-Constrained Trajectory Optimization

Trajectory generation converts the selected discrete route into a smooth flight trajectory. Safe corridors guide position optimization, while separate yaw generation and trajectory checks preserve the required observation orientation and enforce acceptance criteria.

After JTVP selects the next task and observation pose, the trajectory optimizer connects the discrete A* route to the start and goal, constructs geometric safe corridors, and optimizes one continuous trajectory. Safe corridors $\mathcal{K}_r$ are generated from measured point-cloud clearance and Configuration-Space Iterative Regional Inflation (CIRI) [38] geometric seeds, and the resulting piecewise polynomial trajectory is optimized with MINCO [39]. Unknown voxels are treated as blocked, narrow passages are represented by local guide centerlines and centered half-space constraints, and adjacent corridor intersections are checked to prevent a smoothed curve from cutting across a wall.

For R trajectory pieces, the position on piece r is a seventh-degree polynomial,

$$\mathbf{p}_r(\tau) = \sum_{h=0}^{7} \mathbf{a}_{r,h} \tau^h, \qquad \tau \in [0, T_r], \quad T_r > 0. \tag{14}$$

MINCO determines the coefficients from internal interpolation points $\mathbf{U} = (\mathbf{u}_1, \dots, \mathbf{u}_{R-1})$, durations $\mathbf{T}$, and prescribed endpoint position, velocity, acceleration, and jerk. Adjacent pieces share derivatives through order three. Internal points are parameterized in the corresponding corridor intersections. Writing $\mathbf{p} = \mathcal{M}(\mathbf{U}, \mathbf{T})$ for this minimum-snap parameterization, the optimization is

$$\begin{aligned} \min_{\mathbf{U},\, \mathbf{T} > \mathbf{0}} \quad & \sum_{r=1}^{R} \int_0^{T_r} \|\mathbf{p}_r^{(4)}(\tau)\|_2^2 \, d\tau + \rho \sum_{r=1}^{R} T_r \\ & + J_{\text{corr}}(\mathbf{p}, \mathbf{T}) + J_{\text{dyn}}(\mathbf{p}, \mathbf{T}), \end{aligned} \tag{15}$$

where $\rho > 0$ trades trajectory duration against snap energy. Let corridor $\mathcal{K}_r$ have normalized half-space constraints $\mathbf{n}_{r,f}^\mathsf{T} \mathbf{p} + b_{r,f} \le 0$. The soft penalties have the explicit form

$$\begin{aligned} J_{\text{corr}} &= \lambda_c \sum_{r,f} \int_0^{T_r} \phi_\mu(\mathbf{n}_{r,f}^\mathsf{T} \mathbf{p}_r + b_{r,f}) \, d\tau, \\ J_{\text{dyn}} &= \sum_r \sum_{d \in \mathcal{D}} \lambda_d \int_0^{T_r} \phi_\mu(g_d(\mathbf{p}_r, \tau)) \, d\tau. \end{aligned} \tag{16}$$

Here, $\mathcal{D}$ indexes enabled dynamic penalties. Translational penalties use $g_d = \|\mathbf{p}_r^{(d)}\|_2^2 - b_d^2$ for derivative orders $d = 1, 2, 3$ and limits b_d on speed, acceleration, and jerk. Enabled body-rate and thrust penalties use the corresponding differential-flatness quantities and configured bounds. The nonnegative

weights carry the units needed to combine these terms. For smoothing parameter $\mu > 0$, the implementation uses the one-sided penalty

$$\phi_\mu(x) = \begin{cases} 0, & x \le 0, \\ (\mu - x/2)(x/\mu)^3, & 0 < x < \mu, \\ x - \mu/2, & x \ge \mu, \end{cases} \tag{17}$$

Penalty integrals use trapezoidal quadrature with at least 40 subdivisions per piece. Endpoint states, MINCO continuity, positive durations, and internal-point corridor membership are imposed by parameterization; whole-piece corridor and dynamic violations are penalized during optimization and checked before acceptance. Optional guide-attraction and regional-height terms supplement this formulation. Snap limits are checked on the resulting trajectory rather than introduced as a separate snap-limit penalty.

Yaw is generated separately from the position polynomial. A direction-following reference is blended toward the required observation yaw, unwrapped, and sampled at intervals no greater than $0.01\,\mathrm{s}$. A rate-limited profile with prescribed start and terminal yaw is then formed and linearly interpolated. A phase too short to satisfy the yaw-rate bound is rejected.

Trajectory validation uses a maximum temporal step of $0.03\,\mathrm{s}$ and a maximum speed-based spatial step of $0.01\,\mathrm{m}$. For a validation interval of duration Δ with speed bound $v_{\max}^{\mathrm{poly}}$ obtained from the polynomial, the distance to the fixed collision point cloud satisfies

$$\min_{\tau} d_{\mathrm{cloud}}(\mathbf{p}(\tau)) \ge \min_{j} d_{\mathrm{cloud}}(\mathbf{p}(\tau_j)) - \tfrac{1}{2} v_{\max}^{\mathrm{poly}} \Delta. \tag{18}$$

This follows from the 1-Lipschitz property of distance and sampling both ends of each interval. The reserved clearance must exceed the collision margin. This bound concerns the supplied collision geometry; finite sampling alone is not a continuous-time guarantee. Corridor, regional-height, derivative, and yaw-rate checks must also pass before acceptance. Replanning retains the committed prefix and regenerates the remaining route.

Only trajectories that pass the geometric, dynamic, and yaw checks are accepted for execution, linking the selected observation pose to a validated flight command.

F. Observation-Driven Semantic Revision and Recovery

Online verification connects visual observations to task-state updates and search recovery. Positive evidence advances execution; negative evidence updates the search according to the task policy, through local viewpoint retries, alternative semantic regions, or a resolved task outcome.

At the selected observation pose, RGB observations acquired during motion contribute semantic evidence, while terminal RGB-D observations provide open-vocabulary detections, depth-based 3D projections, and geometric verification cues. A positive online verification advances the task state. A negative terminal observation is handled according to the task-specific not-found policy: it may trigger another local viewpoint, bounded semantic recovery, a terminal absence result, or an observation-dependent branch.

Open-vocabulary detections are lifted into the global frame using depth and camera pose. Same-category observations are associated with mapped instances through spatial consistency, while unassociated detections are accumulated across views for subsequent scene-graph revision.

For tasks configured for semantic recovery, LogicFly distinguishes three levels of negative evidence. A **pose-level miss** removes the attempted candidate while keeping the task active. **Region-level exhaustion** invokes the language model with the target, mapped room/object inventory, and failed search anchors to propose alternative room–functional-region hypotheses grounded in the hierarchy; the hypotheses must reference existing room and object IDs and pass deterministic validation before becoming geometric observation regions and are ranked using semantic relevance, coverage, and travel cost. When a room-centered observation is insufficient, the recovery search adds reachable boundary viewpoints oriented toward the room walls. A **task-level negative outcome** is produced when the task-specific stopping condition is reached, after which CPTG updates downstream logic. Within semantic recovery, newly covered portions of an unproductive region reduce its belief, shifting search toward alternative hypotheses.

For a semantic recovery region r, let $\mathcal{P}_r$ be its fixed sample set and $\mathcal{O}_{r,n}$ the samples already covered before verification n, initialized as $\mathcal{O}_{r,0} = \varnothing$. With visible samples $\mathcal{V}_r(\mathbf{q}_n)$, define

$$\begin{aligned} \delta_{r,n} &= \frac{|\mathcal{V}_r(\mathbf{q}_n) \setminus \mathcal{O}_{r,n}|}{\max(1, |\mathcal{P}_r|)}, \\ \mathcal{O}_{r,n+1} &= \mathcal{O}_{r,n} \cup \mathcal{V}_r(\mathbf{q}_n). \end{aligned} \tag{19}$$

The task-local confidence $b_{r,0}$ is initialized from semantic relevance: high, medium, low, and fallback hypotheses receive 1.0, 0.6, 0.3, and 0.15. Registration preserves an existing region's accumulated state. If $y_n \in \{0, 1\}$ indicates successful target verification and $Q_r(\mathbf{q}_n)$ is geometric view quality, the update is

$$b_{r,n+1} = \begin{cases} 1, & y_n = 1, \\ b_{r,n}\big[1 - \alpha \delta_{r,n} Q_r(\mathbf{q}_n)\big], & y_n = 0, \end{cases} \tag{20}$$

with detector reliability $\alpha = 0.80$. These scores remain in $[0, 1]$ and are not normalized across regions or interpreted as a posterior probability distribution. Repeated inspection of already covered samples gives $\delta_{r,n} = 0$, avoiding repeated confidence reduction from the same coverage.

Recovery views replace the ordinary terminal score by

$$C_{r,n}^{\mathrm{rec}}(\mathbf{q}) = 1.5(1 - b_{r,n}) + 0.6\widehat{c}_r + 0.8[1 - Q_r(\mathbf{q})]. \tag{21}$$

Here, $\widehat{c}_r$ is the min–max normalized coverage-route cost across proposed regions, set to zero if all costs coincide. The route estimate greedily selects up to four views by newly covered samples, sums their A* travel lengths, and adds $2\,\mathrm{m}$ times the uncovered fraction; an unreachable estimate receives a $10^3\,\mathrm{m}$ sentinel cost. Thus, confidence and coverage affect terminal quality, while travel from the current pose remains in (11).

The recovery procedure is: (1) verify the selected view and update (19)–(20); (2) remove a missed view and retry unused local candidates; (3) on local exhaustion, admit permitted semantic-region hypotheses and rescore using (21); and (4)

TABLE I

BENCHMARK RESULTS GROUPED BY TASK DIFFICULTY AND FLOOR SCOPE. THE CLEAN SCHEDULE CONTAINS 270 SINGLE-FLOOR AND 38 CROSS-FLOOR SINGLE-TARGET EPISODES, AND 364 SINGLE-FLOOR AND 182 CROSS-FLOOR MULTI-TARGET EPISODES. SR AND SPL ARE PERCENTAGES; CT IS AVERAGED OVER SUCCESSFUL EPISODES. DASHES OMIT SPL AND CT WHEN NO EPISODE SUCCEEDS. ONEMAP USES ITS ORDERED-CATEGORY PROJECTION. BOLD DENOTES THE BEST RESULT IN EACH TASK GROUP.

Task type	Method	Single-floor			Cross-floor		
		SR↑ (%)	SPL↑ (%)	CT↓ (s)	SR↑ (%)	SPL↑ (%)	CT↓ (s)
Single-target	SpatialNav	8.15	5.89	22.35	0.00	–	–
	SpatialGPT	10.37	2.69	115.39	0.00	–	–
	MapGPT	27.78	14.35	42.95	18.42	13.08	81.57
	OneMap	52.59	35.45	77.32	21.05	20.30	49.88
	LogicFly	**97.78**	**87.28**	**8.19**	**94.74**	**86.03**	**8.99**
Multi-target	MapGPT	1.10	0.73	96.02	0.00	–	–
	OneMap	12.36	8.16	150.13	6.59	6.59	90.59
	LogicFly	**96.70**	**82.49**	**10.33**	**96.70**	**81.45**	**14.70**

resolve the task when verification succeeds or its declared observation/search policy terminates. The implementation allows at most three alternative hypotheses per request and seven distinct locations per task. A first-viewpoint branch is decided at that designated observation rather than delayed for exhaustive search. Exhausting this budget means "not found within the prescribed search policy", not proof of global nonexistence; only a matching task predicate accepts that negative outcome. An unresolved positive-search goal or an episode timeout remains a failure.

Novel detections are accumulated online through spatially consistent multi-view fusion and materialized in the updated scene graph after execution. Thus, task states and semantic-region beliefs can affect replanning within the current execution, whereas newly created object instances support later use of the revised map.

After each terminal verification, the system updates the task state, candidate poses, and semantic search prior, recomputes $\mathcal{E}_t$, and invokes JTVP from the current robot pose. When a task reaches a task-level outcome, CPTG additionally updates $\mathcal{R}_t$ and evaluates $\mathcal{R}_{\text{cond}}$ to activate or skip downstream conditional tasks. This closes the loop among task logic, 3D observation, geometric motion, and online semantic evidence.

V. EXPERIMENTS

We evaluate LogicFly through simulation, ablation studies, and real-world UAV experiments, covering benchmark performance, the contributions of CPTG, JTVP, and semantic recovery, and practical deployment.

A. Experimental Setup

Evaluation benchmark. We evaluate all methods in the Habitat simulator [40] on six HM3D [41] scenes: 00070, 00166, 00273, 00290, 00337, and 00409. The benchmark contains 152 language instructions, including 61 short single-target instructions, 60 long-horizon instructions confined to a single floor, and 31 long-horizon cross-floor instructions. Using the valid initial-state seeds and two repeated trials, each method is evaluated over 854 episodes. The benchmark covers ordered execution, conditional branches, target absence, failure recovery, spatial relations, and instance disambiguation.

Evaluation metrics. We report task Success Rate (SR), Success weighted by Path Length (SPL), and Completion Time (CT). An episode is considered successful only when all required task outcomes are completed while satisfying the instruction-level ordering, branching, spatial, recovery, and disambiguation constraints. Reaching the correct objects without satisfying the complete instruction is counted as a failure. For an episode e with a defined geometric reference route, the reported path-efficiency metric is

$$\text{SPL}_e = S_e \frac{L_e^{\text{ref}}}{\max\{L_e^{\text{ref}}, L_e, \varepsilon_L\}}, \qquad \varepsilon_L = 10^{-9}\,\text{m}, \tag{22}$$

where $S_e \in \{0, 1\}$ is strict task success and L_e is the sum of three-dimensional distances along the recorded trajectory. The evaluator constructs L_e^{ref} retrospectively using the required tasks selected by the recorded conditional-source observation, retaining instruction precedence and branch ordering. If that source was never observed, all potentially required tasks remain in the reference problem. It uses the same voxel geometry, a finite generated candidate set, A* transition costs, and the joint planner's weighted objective; L_e^{ref} is the length of the resulting route, not a certified shortest path in continuous space or a pure path-length optimum. This reference construction is separate from the online policy and does not provide future branch outcomes to the robot.

For a required target-absence judgment, the frozen search-anchor center is used as the reference endpoint, with the planner's nearest-free-voxel adjustment. These episodes are therefore included when that anchor yields a valid reference route. No unique endpoint or no feasible reference route makes SPL undefined. For applicable episodes, failures and timeouts contribute zero; the denominator is the number of applicable episodes, not the number of successful episodes. The recorded LogicFly results have 854 applicable episodes in Table I, including 54 target-absence episodes, and all 546 multi-target episodes are applicable in Table II. Under the implemented numerical convention, $L_e^{\text{ref}} = 0$ yields zero even if $L_e = 0$;

Fig. 8. Examples of long-instruction execution in Habitat. Each row pairs a natural-language instruction with the corresponding sequence of first-person observations, covering freely ordered subtasks and observation-dependent search or verification.

otherwise (22) lies in $[0, 1]$. Group scores are arithmetic means over their applicable episodes.

Baselines. We compare four map-based and spatial-reasoning methods. OneMap [13] queries a real-time open-vocabulary feature map for multi-object navigation. MapGPT [30] uses map-guided LLM prompting and adaptive route planning. SpatialGPT [31] applies spatial chain-of-thought reasoning over structured memory. SpatialNav [32] uses object-relation scene graphs for zero-shot navigation.

Implementation details. LogicFly first compiles each instruction into a conditional and precedence-constrained task graph, which is subsequently executed by the proposed task-graph runner. Reusable semantic maps are constructed from registered RGB-D observations. The mapping front end uses SAM 3.1 in image mode with a fixed vocabulary of 30 indoor object categories and a confidence threshold of 0.50. Video tracking is disabled. Boxer lifts the accepted masks into 3-D and performs multi-view instance fusion using an IoU threshold of 0.20, an instance-confidence threshold of 0.45, and a minimum of four observations. During task execution, online object evidence is obtained from calibrated RGB-D observations using open-vocabulary recognition and is projected into the world frame using the estimated camera pose and aligned depth.

Room partitioning uses a $0.05\,\mathrm{m}$ horizontal grid, a 3×3 wall-closing neighborhood, an interior-clearance threshold of $\tau_c = 0.45\,\mathrm{m}$, and a minimum core area of $A_{\min} = 0.04\,\mathrm{m}^2$. Horizontal-surface support requires $|n_z| \geq 0.90$ and a projected support distance within $0.12\,\mathrm{m}$. Free-space support requires observations at at least two of four heights, 0.45, 0.85, 1.25, and $1.65\,\mathrm{m}$ above the floor, with a $0.09\,\mathrm{m}$ proximity tolerance. Depth-ray support requires at least three source frames, and trajectory support uses a $0.15\,\mathrm{m}$ radius. A region can merge into its sole neighbor only when the boundary wall fraction is below 0.20 and the boundary does not touch unsupported open space. Room classification uses up to two selected RGB views and a confidence threshold of 0.65; insufficient evidence yields an unknown label. Online fusion associates same-category novel detections within $0.75\,\mathrm{m}$ and requires at least two supporting observations before materializing a new instance.

The joint task-and-viewpoint planner retains up to three representative viewpoints per location and uses a receding planning horizon of $H = 3$ tasks. In (8), $(\beta_d, \beta_Q, \beta_z, \beta_r, \beta_b) = (1, 1, 0.10, 0.15, 0.15)$. In (11), $(\eta_L, \eta_T, \eta_V) = (1, 0.25, 2)$, with reference scales $\ell_0 = 1\,\mathrm{m}$ and $\tau_0 = 1\,\mathrm{s}$. The nominal path speed, climb speed, and yaw rate are $1.0\,\mathrm{m/s}$, $0.5\,\mathrm{m/s}$, and $1.0\,\mathrm{rad/s}$, respectively. A collision safety margin of $0.10\,\mathrm{m}$ is enforced during trajectory generation in both simulated and real-world experiments. All language-model components use Qwen3.8-Flash. Semantic recovery generates at most three alternative hypotheses per request and permits at most seven distinct location hypotheses per task, with up to four viewpoints retained per location. Local reinspection exhausts the finite candidate pool before advancing to another permitted location. High-, medium-, and low-relevance region priors are 1.0, 0.6, and 0.3, respectively; the fallback prior is 0.15. Negative-evidence updates use detector reliability 0.80 and newly covered samples, with scoring weights of 1.5, 0.6, and 0.8 for belief deficiency, normalized coverage cost, and view-quality deficiency, respectively. Task-specific branch and not-found policies determine whether a negative observation immediately resolves a task or invokes bounded recovery. All perception, planning, and recovery parameters are fixed across scenes and episode repetitions.

Runtime protocol. Each evaluation process is subject to

a fixed time limit of 240 s. A timeout or execution error is counted as a failed episode. For LogicFly, CT is measured using its native completion timestamp and excludes one-time map preparation. For the comparison methods, CT is measured from process startup to task completion. CT is averaged over successful episodes only.

B. Comparison with Baseline Methods in Simulation

We compare LogicFly with HM3D adaptations of OneMap [13], MapGPT [30], SpatialGPT [31], and SpatialNav [32]. All methods run on the same 854-episode clean schedule. OneMap retains its native ordered-category projection, which omits relations, conditional branches, verified absence, recovery, and return constraints. All baselines and LogicFly are evaluated using the same frozen strict full-task evaluator. SpatialGPT and SpatialNav do not successfully complete any cross-floor episode, resulting in an SR of 0%; therefore, their remaining cross-floor metrics are not reported.

As shown in Table I, LogicFly outperforms the compared methods across target counts and floor scopes. On multi-target tasks, it maintains 96.70% SR on both floor scopes, with SPL of 82.49% and 81.45% and CT of 10.33 s and 14.70 s for single-floor and cross-floor episodes, respectively. SpatialGPT and SpatialNav complete no multi-target episodes and are omitted from that block. Overall, LogicFly completes 828/854 episodes (96.96% strict full-task SR, 0.8394 SPL).

Execution with more subtasks. In an additional cross-floor simulation in scene 00337, LogicFly completes all 15 subtasks of a single natural-language instruction in one mission. With the same fixed planning horizon $H = 3$, language parsing and native navigation share a 240 s deadline; the complete run takes 148.96 s, including 59.24 s for parsing and 89.72 s for navigation. The 95th-percentile planning-call latency is 7.95 s. This run demonstrates completion beyond the planning-window size without a timeout; it is a single-run scalability demonstration rather than an estimate of the maximum supported task count.

Figure 8 presents a qualitative execution example in a three-floor Habitat environment. LogicFly completes the cross-floor route and all required subtasks, with terminal views providing visual evidence for each successful outcome.

All baselines retain their native planning, action, and stopping interfaces. OneMap receives the protocol-defined ordered-category projection, whereas MapGPT, SpatialGPT, and SpatialNav receive the frozen task specification through their native interfaces. Timeouts and algorithm failures remain unsuccessful terminal outcomes.

C. Ablation Studies

CPTG and JTVP. We evaluate CPTG and JTVP on the 546 multi-target episodes from the frozen six-scene HM3D schedule, covering all multi-target task types. The analysis contains 1638 full-mission evaluation records, reusing the corresponding native benchmark runs for the full method. All three variants use outer-run completion times for CT, including the reused records.

TABLE II
ABLATION RESULTS ON THE MULTI-TARGET SUBSET OF 546 EPISODES. SR AND SPL ARE PERCENTAGES. CT USES THE SAME OUTER-RUN TIMING FIELD FOR ALL VARIANTS AND IS AVERAGED OVER SUCCESSFUL EPISODES.

Variant	SR↑ (%)	SPL↑ (%)	CT↓ (s)
w/o CPTG	64.84	58.45	35.63
w/o JTVP	**96.70**	76.82	12.62
LogicFly	**96.70**	**82.15**	**11.79**

TABLE III
SEMANTIC-RECOVERY ABLATION ON 10 CROSS-REGION TASKS, WITH 60 EPISODES PER METHOD. SR AND SPL ARE PERCENTAGES; CT IS AVERAGED OVER SUCCESSFUL EPISODES. A DASH DENOTES NO SUCCESSFUL EPISODE.

Variant	SR↑ (%)	SPL↑ (%)	CT↓ (s)
w/o recovery	0.00	0.00	–
LogicFly	**80.00**	**53.46**	14.03

All variants share the native Habitat runner, online open-vocabulary verification, strict evaluator, scene representations, start poses, motion constraints, and termination settings. *w/o CPTG* removes task dependencies, conditional rules, and task-conditioned candidate generation, retaining target labels, coarse room/floor scopes, generic observation constraints, and fixed language order. *w/o JTVP* retains CPTG but fixes task order and uses only each task's primary viewpoint. *LogicFly* jointly optimizes task ordering and viewpoints. The evaluator enforces required outcomes and ordering, forbids visits to skipped branches, and checks spatial/reference constraints.

Table II shows that removing CPTG reduces SR to 64.84% and SPL to 58.45%, while increasing successful-task CT to 35.63 s. Failures to enforce precedence and observation-dependent branches lead to incorrect ordering, unnecessary visits, and repeated recovery, highlighting CPTG's contribution to full-task correctness. Retaining CPTG with fixed order and primary viewpoints completes 528/546 episodes (96.70% SR) without infrastructure failures, with 76.82% SPL and 12.62 s CT. JTVP preserves this SR while raising SPL by 5.33 percentage points to 82.15% and reducing CT to 11.79 s (approximately 6.6%). Thus, CPTG primarily supports instruction correctness, while JTVP improves path and time efficiency. The 30 w/o CPTG timeouts remain in the 546-episode SR and SPL denominators, with zero SPL.

Semantic recovery. We further evaluate 10 cross-region target-search tasks across the same six scenes, covering both same-floor and cross-floor recovery. Each task starts with an incorrect target-location prior in a region without the target. True target-category entries are removed from the runtime semantic map, while the physical scene and other map entries are preserved. Before evaluation, all initial candidate views are checked for target absence, and reachable views in another region are verified to detect a real target. Three fixed starting states and two repetitions yield 60 episodes per method, or 120 newly executed episodes. The ablated variant retains CPTG, JTVP, local multi-view reinspection, and the same perception and motion settings, but disables semantic-region recovery and

its frontier fallback. Both groups use the same perturbed maps and starting states, a 240 s budget, and native caching.

Success requires exhausting the initial local search, leaving that region, and completing the task with RGB-D target evidence associated with a true same-category instance within 1 m. For SPL, the independently frozen A* reference route visits $\min(3, n_A)$ distinct initial viewpoints before a true-target candidate, where n_A is the initial candidate count. CT uses successful episodes' outer-process wall-clock times, including initialization, planning, and perception.

Table III shows that LogicFly succeeds in 48 of 60 episodes, whereas removing semantic recovery reduces SR to zero. All 60 full-method episodes trigger recovery after local search exhaustion; eight tasks succeed in every starting state and repetition. The full method achieves an SPL of 53.46% and a successful-task CT of 14.03 s. Of its 12 unsuccessful episodes, six end without resolving the search and six report completion without passing the true-target geometric check. All 120 episodes finish without infrastructure failures or timeouts. These results demonstrate that semantic recovery enables cross-region search after local failure and substantially improves task completion when the initial target-location prior is incorrect.

D. Evaluation in Real-World Environments

Experimental platform and configuration. The experiments use a custom-built quadrotor equipped with an NVIDIA Jetson Orin NX (16 GB), a PX4 flight controller, a Livox Mid-360 LiDAR, and an Intel RealSense D435 RGB-D camera. The camera provides RGB-D observations at a resolution of 640×480, while FAST-LIO [42] provides onboard localization. The maximum flight speed, acceleration, and yaw rate are set to 1.0 m/s, 1.0 m/s^2, and 1.0 rad/s, respectively. During flight, LogicFly performs task planning and trajectory generation online using the semantic–geometric map and the current UAV state. Visual verification results update task states and guide subsequent planning and recovery.

Experimental environments and tasks. We conduct 10 flight experiments in laboratory, office, and real-world home environments, using different starting positions and task instructions. The experiments cover single-target search and multi-target inspection in settings with narrow doorways, multi-room layouts, and furniture-induced occlusions. These settings require the UAV to navigate between regions and reach observation poses suitable for the requested checks.

Trajectory execution in constrained spaces. The real-world home environment includes narrow doorways, with the narrowest measuring approximately 0.60 m in width. A* provides geometric guidance through constrained spaces, safe corridors define the feasible trajectory space, and MINCO converts the discrete route into a smooth piecewise-polynomial trajectory by balancing snap minimization and trajectory duration. Clearance and dynamic-limit checks are applied before trajectory execution, while yaw planning aligns the camera with the required observation direction. The recorded flights demonstrate the coordinated operation of geometric planning,

Fig. 9. Representative real-world UAV executions. From top to bottom: a multi-target household inspection, a three-target laboratory inspection, a ball-search task at a pool table, and a cup-search task at a lounge table. Each row shows successive third-person views; yellow circles mark the UAV and green boxes highlight selected target objects.

trajectory optimization, and flight control around furniture and within narrow passages.

Spatial constraints and task logic. The instructions specify targets on tables, beneath furniture, and on upper shelves, requiring observation positions and viewing directions that account for both targets and their reference objects. Multi-target missions also include mandatory precedence and observation-dependent branches. For example, the home-inspection instruction requires a kitchen inspection before subsequent checks and specifies an alternative search location if the cup is absent from its expected position. These tasks test the translation of spatial relations, task dependencies, and conditional rules into physical navigation and inspection actions.

Online recovery after an unsuccessful observation. Figure 1 presents a four-task mission involving recovery beyond the initial target location. The UAV first inspects the expected laptop location, where the target is not observed. Semantic recovery then directs the search to the reading room, where the laptop is found, after which the UAV continues with the remaining inspections, including the toolbox and microwave. The recorded trajectory and observation sequence document the initial unsuccessful inspection, the search in an alternative semantic region, and subsequent task execution. This example demonstrates online adaptation when the initial target-location prior is incorrect.

Results and failure analysis. Figure 9 shows representative multi-target inspections and single-target searches. Across the 10 experiments, LogicFly successfully completes eight missions, yielding a mission success rate of 80%. Both unsuccessful missions result from target misdetections, indicating that perception errors can affect visual verification and task-outcome assessment. The successful executions support the feasibility of online language-guided aerial inspection across the tested indoor settings, including spatially constrained observations, multi-target execution, and search recovery. The failures identify visual perception and verification reliability as remaining limitations.

VI. Conclusion

We presented **LogicFly**, a language-guided aerial navigation framework built around CPTG and JTVP. CPTG separates precedence and observation-triggered branches; JTVP selects task order and observation poses with 3D path and terminal costs within a bounded window. Verification and constrained recovery update subsequent decisions.

Across 854 episodes, LogicFly achieves 96.96% strict full-task SR and 0.8394 SPL, including 96.70% multi-target SR. Ablations verify CPTG's role in complex task logic. Compared with fixed task order and primary viewpoints, JTVP raises SPL from 76.82% to 82.15% and reduces successful-task CT from 12.62 s to 11.79 s at unchanged SR. Semantic recovery achieves 80.0% SR versus 0% without it on cross-region tasks, demonstrating its effectiveness under incorrect target-location priors. Deployment on a custom quadrotor further demonstrates multi-target execution with logical dependencies in real indoor rooms. These results advance VLN toward efficient closed-loop execution of logically complex missions.

References

[1] P. Anderson *et al.*, "Vision-and-language navigation: Interpreting visually-grounded navigation instructions in real environments," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 3674–3683.

[2] J. Krantz, E. Wijmans, A. Majumdar, D. Batra, and S. Lee, "Beyond the Nav-Graph: Vision-and-language navigation in continuous environments," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, ser. Lecture Notes Comput. Sci., vol. 12373, 2020, pp. 104–120.

[3] M. Shridhar *et al.*, "ALFRED: A benchmark for interpreting grounded instructions for everyday tasks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 10 740–10 749.

[4] X. Song, W. Chen, Y. Liu, W. Chen, G. Li, and L. Lin, "Towards long-horizon vision-language navigation: Platform, benchmark and method," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2025, pp. 12 078–12 088.

[5] S. Wani, S. Patel, U. Jain, A. X. Chang, and M. Savva, "MultiON: Benchmarking semantic map memory using multi-object navigation," in *Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 9700–9712.

[6] M. Khanna *et al.*, "GOAT-Bench: A benchmark for multi-modal lifelong navigation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 16 373–16 383.

[7] Z. Dai *et al.*, "Optimal scene graph planning with large language model guidance," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024, pp. 14 062–14 069.

[8] C. Huang, O. Mees, A. Zeng, and W. Burgard, "Visual language maps for robot navigation," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2023, pp. 10 608–10 615.

[9] A. Werby, C. Huang, M. Büchner, A. Valada, and W. Burgard, "Hierarchical open-vocabulary 3D scene graphs for language-grounded robot navigation," in *Proc. Robot.: Sci. Syst. (RSS)*, 2024.

[10] X. Huang *et al.*, "MSGNav: Unleashing the power of multi-modal 3D scene graph for zero-shot embodied navigation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2026, pp. 37 154–37 163.

[11] K. Rana, J. Haviland, S. Garg, J. Abou-Chakra, I. Reid, and N. Sünderhauf, "SayPlan: Grounding large language models using 3D scene graphs for scalable robot task planning," in *Proc. 7th Conf. Robot Learn. (CoRL)*, ser. Proc. Mach. Learn. Res., vol. 229, 2023, pp. 23–72.

[12] W. Huang *et al.*, "Inner monologue: Embodied reasoning through planning with language models," in *Proc. 6th Conf. Robot Learn. (CoRL)*, ser. Proc. Mach. Learn. Res., vol. 205, 2023, pp. 1769–1782.

[13] F. L. Busch, T. Homberger, J. Ortega-Peimbert, Q. Yang, and O. Andersson, "One map to find them all: Real-time open-vocabulary mapping for zero-shot multi-object navigation," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2025, pp. 14 835–14 842.

[14] A. Ku, P. Anderson, R. Patel, E. Ie, and J. Baldridge, "Room-Across-Room: Multilingual vision-and-language navigation with dense spatiotemporal grounding," in *Proc. Conf. Empir. Methods Natural Lang. Process. (EMNLP)*, 2020, pp. 4392–4412.

[15] Y. Qi *et al.*, "REVERIE: Remote embodied visual referring expression in real indoor environments," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 9982–9991.

[16] F. Zhu, X. Liang, Y. Zhu, Q. Yu, X. Chang, and X. Liang, "SOON: Scenario oriented object navigation with graph-based exploration," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, pp. 12 689–12 699.

[17] W. Hao, C. Li, X. Li, L. Carin, and J. Gao, "Towards learning a generic agent for vision-and-language navigation via pre-training," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 13 137–13 146.

[18] Z. Wang *et al.*, "Scaling data generation in vision-and-language navigation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2023, pp. 12 009–12 020.

[19] S. Chen, P.-L. Guhur, C. Schmid, and I. Laptev, "History aware multimodal transformer for vision-and-language navigation," in *Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 5834–5847.

[20] S. Chen, P.-L. Guhur, M. Tapaswi, C. Schmid, and I. Laptev, "Think global, act local: Dual-scale graph transformer for vision-and-language navigation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 16 537–16 547.

[21] C. H. Song, J. Kil, T.-Y. Pan, B. M. Sadler, W.-L. Chao, and Y. Su, "One step at a time: Long-horizon vision-and-language navigation with milestones," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 15 482–15 491.

[22] J. Krantz *et al.*, "Iterative vision-and-language navigation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2023, pp. 14 921–14 930.
[23] A. Rosinol, A. Gupta, M. Abate, J. Shi, and L. Carlone, "3D dynamic scene graphs: Actionable spatial perception with places, objects, and humans," in *Proc. Robot.: Sci. Syst. (RSS)*, 2020.
[24] N. Hughes, Y. Chang, and L. Carlone, "Hydra: A real-time spatial perception system for 3D scene graph construction and optimization," in *Proc. Robot.: Sci. Syst. (RSS)*, 2022.
[25] S. Peng, K. Genova, C. Jiang, A. Tagliasacchi, M. Pollefeys, and T. Funkhouser, "OpenScene: 3D scene understanding with open vocabularies," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2023, pp. 815–824.
[26] Q. Gu *et al.*, "ConceptGraphs: Open-vocabulary 3D scene graphs for perception and planning," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024, pp. 5021–5028.
[27] D. Shah, B. Osiński, B. Ichter, and S. Levine, "LM-Nav: Robotic navigation with large pre-trained models of language, vision, and action," in *Proc. 6th Conf. Robot Learn. (CoRL)*, ser. Proc. Mach. Learn. Res., vol. 205, 2023, pp. 492–504.
[28] B. Yu, H. Kasaei, and M. Cao, "L3MVN: Leveraging large language models for visual target navigation," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2023, pp. 3554–3560.
[29] G. Zhou, Y. Hong, and Q. Wu, "NavGPT: Explicit reasoning in vision-and-language navigation with large language models," in *Proc. AAAI Conf. Artif. Intell.*, vol. 38, no. 7, 2024, pp. 7641–7649.
[30] J. Chen, B. Lin, R. Xu, Z. Chai, X. Liang, and K.-Y. K. Wong, "MapGPT: Map-guided prompting with adaptive path planning for vision-and-language navigation," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL), Vol. 1: Long Papers*, 2024, pp. 9796–9810.
[31] Z. Jiang and X. Wang, "SpatialGPT: Zero-shot vision-and-language navigation via spatial CoT over structured spatial memory," in *Proc. 33rd ACM Int. Conf. Adv. Geographic Inf. Syst. (SIGSPATIAL)*, 2025, pp. 423–435.
[32] J. Zhang, Z. Li, S. Wang, X. Shi, Z. Wei, and Q. Wu, "SpatialNav: Leveraging spatial scene graphs for zero-shot vision-and-language navigation," *arXiv preprint arXiv:2601.06806*, 2026.
[33] N. P. Bhatt *et al.*, "VLN-Zero: Rapid exploration and cache-enabled neurosymbolic vision-language planning for zero-shot transfer in robot navigation," *arXiv preprint arXiv:2509.18592*, 2025.
[34] Y. Qiao *et al.*, "Open-Nav: Exploring zero-shot vision-and-language navigation in continuous environment with open-source LLMs," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2025, pp. 6710–6717.
[35] A. Ray, C. Bradley, L. Carlone, and N. Roy, "Task and motion planning in hierarchical 3D scene graphs," *arXiv preprint arXiv:2403.08094*, 2024.
[36] Y. Zhang, X. Chen, C. Feng, B. Zhou, and S. Shen, "FALCON: Fast autonomous aerial exploration using coverage path guidance," *IEEE Trans. Robot.*, vol. 41, pp. 1365–1385, 2025.
[37] D. DeTone *et al.*, "Boxer: Robust lifting of open-world 2D bounding boxes to 3D," *arXiv preprint arXiv:2604.05212*, 2026.
[38] Y. Ren *et al.*, "Safety-assured high-speed navigation for MAVs," *Sci. Robot.*, vol. 10, no. 98, 2025, art. no. eado6187.
[39] Z. Wang, X. Zhou, C. Xu, and F. Gao, "Geometrically constrained trajectory optimization for multicopters," *IEEE Trans. Robot.*, vol. 38, no. 5, pp. 3259–3278, 2022.
[40] M. Savva *et al.*, "Habitat: A platform for embodied AI research," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 9339–9347.
[41] S. K. Ramakrishnan *et al.*, "Habitat-Matterport 3D Dataset (HM3D): 1000 large-scale 3D environments for embodied AI," in *Proc. Neural Inf. Process. Syst. Track Datasets Benchmarks*, vol. 1, 2021.
[42] W. Xu and F. Zhang, "FAST-LIO: A fast, robust LiDAR-inertial odometry package by tightly-coupled iterated Kalman filter," *IEEE Robot. Autom. Lett.*, vol. 6, no. 2, pp. 3317–3324, 2021.